

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS
(NIVEL 2 (INTERMEDIO))

Módulo 01: Estructuras de Datos Avanzadas

«Pilas LIFO, Colas FIFO y Árboles Jerárquicos»

Estructuración de datos de alto rendimiento: Pilas (Stack - LIFO), Colas (Queue / deque - FIFO), Conjuntos (sets) y árboles binarios de búsqueda (BST).

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Intermedio | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Estructuras Lineales y Jerárquicas en Memoria	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Comparativa Visual: Pila (LIFO) vs Cola (FIFO)	Pág. 5
Capítulo 3	Implementación Práctica en Python: Validador de Paréntesis Balanceados con Pilas	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas y Optimización de Estructuras	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender las disciplinas de acceso LIFO y FIFO y el coste temporal $O(1)$ vs $O(n)$ en memoria.



Competencia Práctica

Implementar pilas para validación sintáctica, colas para buffers de tareas y árboles binarios para búsqueda rápida.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Estructuras Lineales y Jerárquicas en Memoria

Las listas básicas no siempre son la estructura óptima cuando la velocidad de inserción y extracción en los extremos es crítica.

☀ **Metáfora Central: Pilas LIFO, Colas FIFO y Árboles Jerárquicos**

Una Pila (Stack) es como una pila de platos: el último que colocas arriba es el primero que lavas (LIFO: Last In, First Out). Una Cola (Queue) es como la fila del supermercado: el primero que llega es el primero en ser atendido (FIFO: First In, First Out).

Principios Teóricos y Modelo Mental

`collections.deque` en Python permite inserciones y extracciones $O(1)$ tanto por la izquierda como por la derecha, a diferencia de `list.pop(0)` que cuesta $O(n)$.

Los conjuntos (sets) implementan álgebra de conjuntos (unión, intersección, diferencia) con consultas $O(1)$ y garantizan elementos únicos.

⚡ **Regla de Oro en Python**

Para colas FIFO de alto rendimiento en Python, utiliza siempre `collections.deque` en lugar de listas estándar.

2. Comparativa Visual: Pila (LIFO) vs Cola (FIFO)

Mecanismos de inserción (push/enqueue) y extracción (pop/dequeue) en memoria:



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Operación Push / Enqueue: Ingreso de nuevo elemento.	Elemento en memoria
2. Evaluación	En Stack (LIFO): Se coloca en el tope y se extrae del tope.	Último en entrar = Primero en salir
3. Transformación	En Queue (FIFO): Se ingresa por la cola y se extrae por la cabeza.	Primero en entrar = Primero en salir
4. Retorno / Salida	Árboles: Ramifican decisiones jerárquicas izquierda/derecha.	Acceso logarítmico $O(\log n)$



Visualización Mental

Las pilas gestionan llamadas de funciones y el botón Deshacer (Ctrl+Z); las colas gestionan mensajes y colas de impresión.

3. Validador de Paréntesis Balanceados con Pilas

Algoritmo clásico de entrevistas técnicas implementado con una Pila LIFO:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
def validar_parentesis(expresion: str) -> bool:
    pila: list[str] = []
    pares = {"(": ")", "{": "}", "[": "]"

    for char in expresion:
        if char in pares.values():
            pila.append(char) # Push
        elif char in pares:
            if not pila or pila.pop() != pares[char]: # Pop & Check
                return False

    return len(pila) == 0

# Pruebas
print(validar_parentesis("{[()()]}")) # True
print(validar_parentesis("{[(])}")) # False
```

Análisis del Código Fuente

El algoritmo apila los caracteres de apertura y los desapila al encontrar cierres, garantizando correspondencia simétrica en $O(n)$.

4. Gotchas y Optimización de Estructuras

Errores de rendimiento habituales al elegir estructuras de datos:

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar `list.pop(0)` para implementar una cola; obliga a desplazar todos los elementos restantes en memoria generando complejidad $O(n)$.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
cola = []  
cola.insert(0, item) #  $O(n)$  en cada inserción
```

✓ Patrón Pythonic / Correcto

```
from collections import deque  
cola = deque()  
cola.append(item) #  $O(1)$  instantáneo
```

🛡 Consejo de Resiliencia en Producción

Usa sets para eliminar duplicados de una lista en una sola operación: `unicos = list(set(datos))`.

5. Resumen Ejecutivo y Conclusiones

Dominas las estructuras de datos fundamentales para diseñar software de alta concurrencia y algoritmos eficientes.



Logro Alcanzado en esta Clase

Capacidad para elegir la estructura de datos óptima según los requerimientos de tiempo y espacio.

Notas del Instructor y Siguientes Pasos

En el siguiente módulo aprenderemos Búsqueda Binaria, Algoritmos de Ordenamiento y Notación Big-O.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Implementa una cola de prioridad utilizando el módulo `heapq` de Python para despachar tareas según su urgencia.